

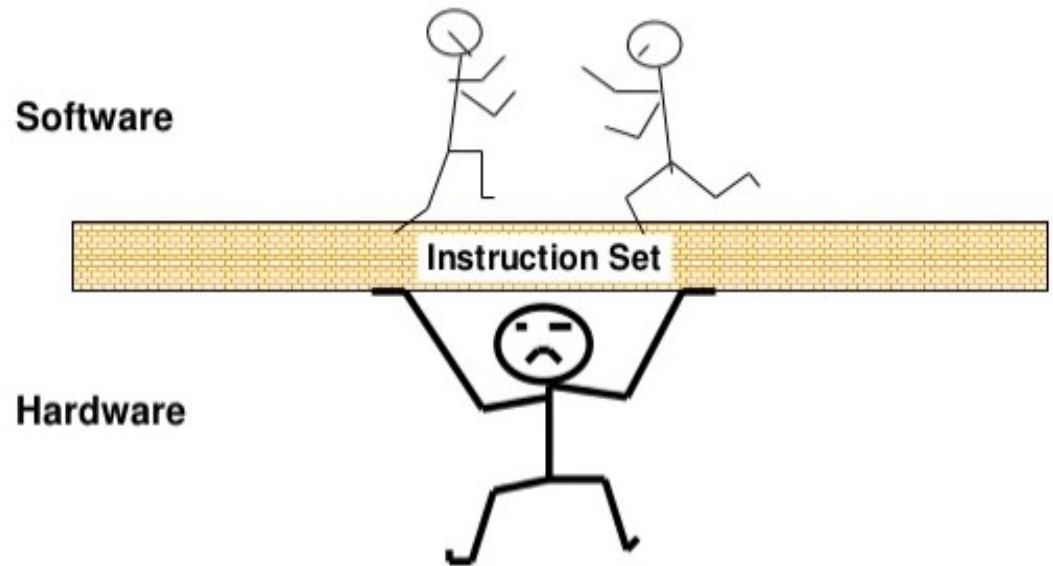
CS1CS Computer Systems Architecture
Instruction – machine language 4
Week 7 lecture 2 of autumn semester 2025

Lectured by

Dr. Hong Wei

h.wei@reading.ac.uk

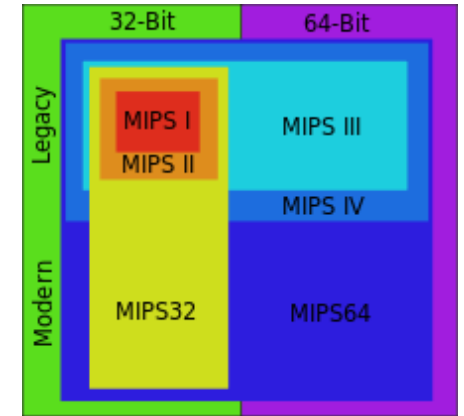
Room 133 in Polly Vacher



(Figure: courtesy of the Internet)

In this lecture

- The following contents will be delivered
 - A model machine: DLX architecture and its instruction set
- The learning outcomes are listed below
 - To be able to describe the DLX architecture with regards to registers, data type, and addressing mode
 - To be able to articulate R-type, I-type, and J-type instruction format used in the DLX architecture
 - Using examples, explain data manipulations in registers and memory cells in various operations, such as load/store, branch/jump, arithmetic/logic, floating point operations



A model machine

DLX Architecture: educational edition of MIPS32/RISC

* MIPS: Million Instruction Per Second

The full set of DLX instructions can be found in the following sources.

1. A book written by Sailer and Kaeli, The DLX Instruction Set Architecture Handbook
2. <http://electro.fisica.unlp.edu.ar/arq/downloads/Software/WinDLX/DLXinst.html>

General introduction

- Simple load-store (register-register) instruction set
- Pipelining for efficiency (the next topic in the module)
- Efficient as a compiler target: **fixed length encoding**, easy to decode
- 32 32-bit general purpose registers (GPR), and 32 32-bit floating point registers
- **Data size**: 8-bit byte, 32-bit integer; 32-bit single precision floating point and 64-bit double precision floating point
- **2 addressing modes**: displacement and immediate.

DLX architecture: registers, data type and addressing modes

- Registers of DLX
 - 32 32-bit general-purpose registers (GPRs), named R0, R1, ..., R31.
 - A set of floating-point registers (FPRs)
32 single-precision registers (F0, F1, ..., F31);
even-odd pairs holding double precision (64-bit) values (F0, F2, ..., F30).
Both 32-bit and 64-bit operations are provided.
The value of R0 is always 0.

- Data types of DLX
 - 32-bit integer (8-bit byte and 16-bit half word are loaded into registers as 32-bit integer with zero filling.)
 - 32-bit single precision floating point
 - 64-bit double precision floating point
- Addressing modes of DLX
 - Byte addressable in **Big Endian** mode with a 32-bit address; **Memory accesses must be aligned.**
 - **Immediate** with 16-bit field
 - **Displacement** with 16-bit field (**register indirect** can be achieved by putting 0 in the 16-bit displacement field; **absolute addressing mode** can be achieved by using register R0 as the base register.)

There could be 4 addressing modes for DLX.

Displacement addressing mode to other addressing modes in DLX

Example: LW R1, 400(R2)

- Register indirect address, when the displacement offset is 0

LW R1, (R2)

- Absolute addressing when $R2=R0$;

LW R1, 400(R0)

DLX architecture: operations

There are four broad classes of operations.

- load and store;
- arithmetic and logical (ALU) operations;
- branch and jump;
- floating-point operations

DLX architecture: instruction format

- Fixed length (32-bits) instruction format
- Encoding addressing modes into opcode
- (Easy to pipeline and decode – why?)
- 6-bit primary opcode
- Three types of instruction format
 - I-type: load-store, branch, ...
 - R-type: add, shift, ...
 - J-type: jump, ...

(1) I-type instruction format

6	5	5	16
Opcode	rs1	rd	immediate

- load and store instructions
- $rd \leftarrow (rs1 \text{ } op \text{ } immediate)$
rs1: source register; rd: destination register
- conditional branch instruction
(rs1 is register; rd unused)
- jump register, jump and link register
(rd=0, rs1=destination, immediate=0)

Examples of I-type instruction format – 1

- Load-store instruction: LW R2, 250(R1)

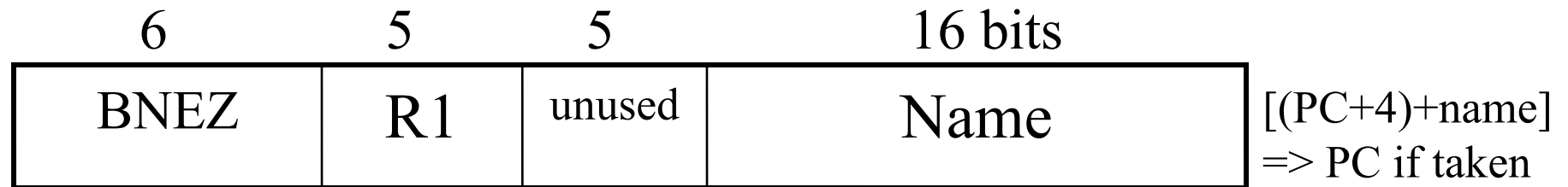
6	5	5	16 bits
LW (displacement)	R1	R2	250

- Immediate ALU instruction: ADDI R2, R1, #100

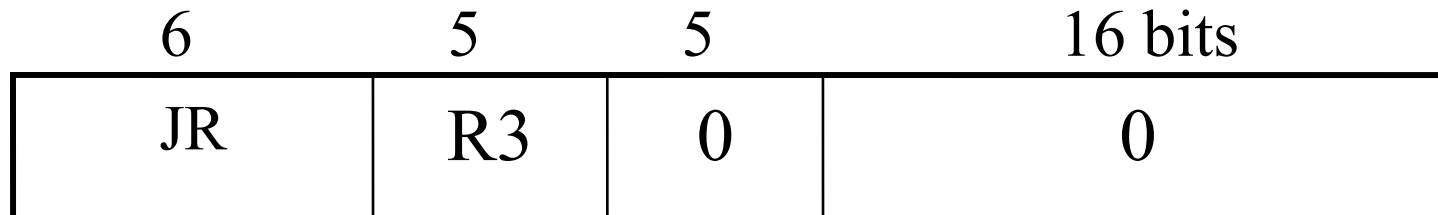
6	5	5	16 bits
ADDI (immediate)	R1	R2	100

Examples of I-type instruction format – 2

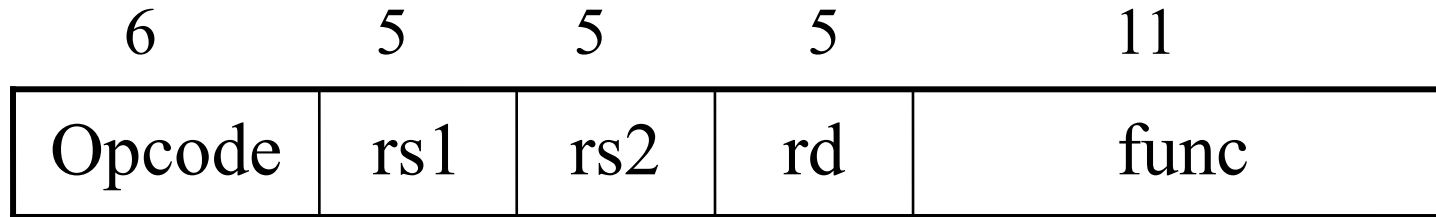
- Conditional branch instruction: BNEZ R1, name



- Jump register instruction: JR R3



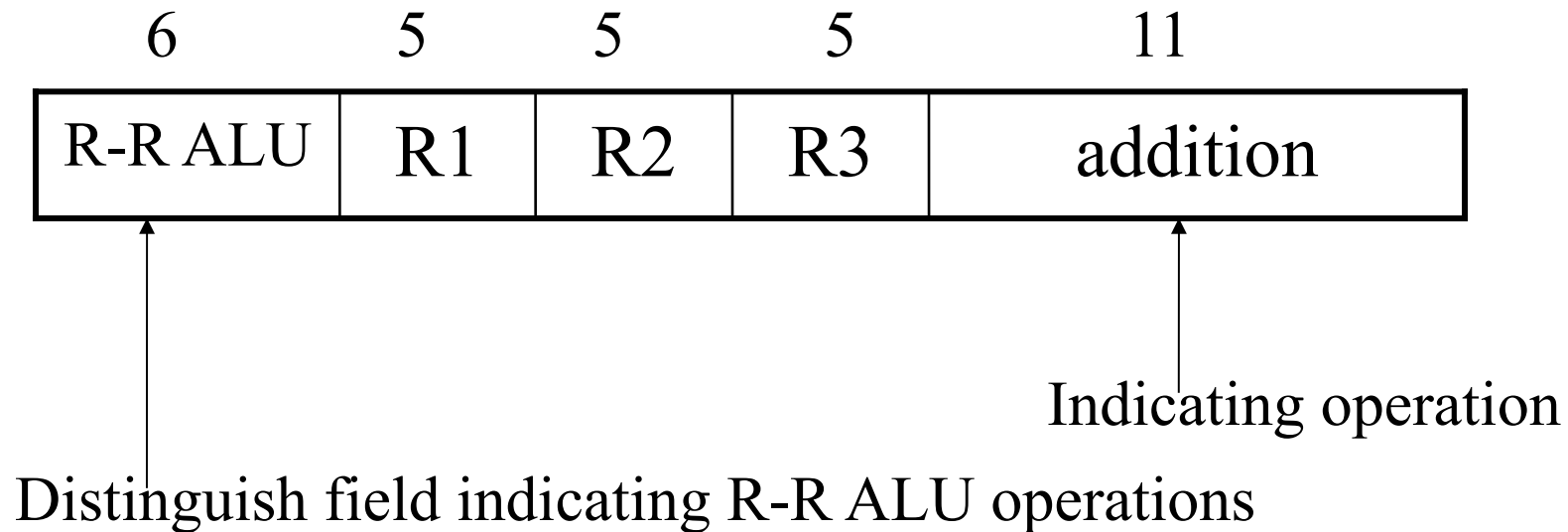
(2) R-type instruction format



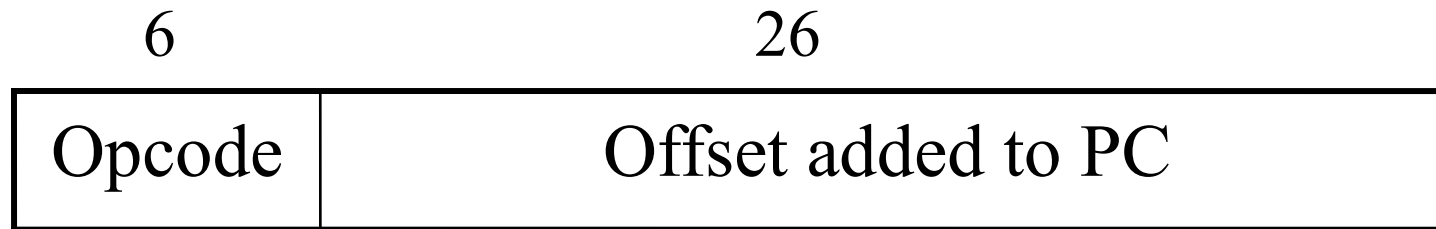
- Register-register ALU operations: $rd \leftarrow (rs1 \text{ func } rs2)$
ALU operation: arithmetic or logical operation
- Function encodes the data path operation: add, sub,...
- Read/write special registers and moves
- Operations only in registers, not involving memory

Example of R-type instruction format

- AL operations: Add R3, R1, R2



(3) J-type instruction format

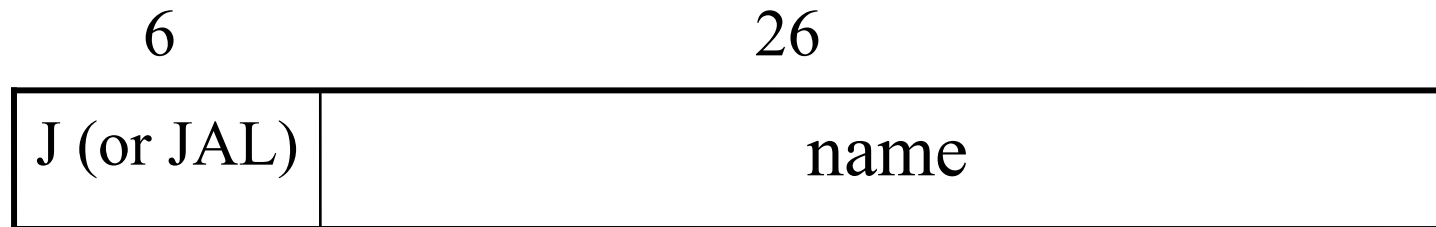


PC: Program Counter register, which means instruction address register to hold the address of the next instruction to be executed

- Jump, **jump and link (used for a function call)**
- Trap and return from exception

Example of J-type instruction format

- Jump instruction: **J name**
- Jump and link instruction: **JAL name**



- Transfer the control to an address specified by “name” unconditionally.
 $[(PC+4)+name] \Rightarrow PC$, where $PC+4$ is called as a delayed slot.
- For JAL, the returned address after the function call is saved in the link register (LR – R31 in DLX).

Examples of instructions

(1) Load and store instructions

instruction	name	meaning
LW R1, 30(R2)	Load word	$\text{Regs}[\text{R1}] \xleftarrow{32} \text{Mem}[30 + \text{Regs}[\text{R2}]]$
LD F0, 50(R3)	Load double	$\text{Regs}[\text{F0}] \text{##} \text{Regs}[\text{F1}] \xleftarrow{64} \text{Mem}[50 + \text{Regs}[\text{R3}]]$
SW 100(R4), R3	Store word	$\text{Mem}[100 + \text{Regs}[\text{R4}]] \xleftarrow{32} \text{Regs}[\text{R3}]$
SD 40(R3), F0	Store double	$\text{Mem}[40 + \text{Regs}[\text{R3}]] \xleftarrow{32} \text{Regs}[\text{F0}]$ $\text{Mem}[44 + \text{Regs}[\text{R3}]] \xleftarrow{32} \text{Regs}[\text{F1}]$

(2) ALU instructions

instruction	name	meaning
ADD R1, R2, R3	Add	$\text{Regs}[\text{R1}] \leftarrow_{32} \text{Regs}[\text{R2}] + \text{Regs}[\text{R3}]$
ADDI R1, R2, #3	Add immediate	$\text{Regs}[\text{R1}] \leftarrow_{32} \text{Regs}[\text{R2}] + 3$

(3) Control flow instructions

JR R3	Jump register	$\text{PC} \leftarrow \text{Regs}[\text{R3}]$
BEQZ R4, name	Branch equal 0	If $(\text{regs}[\text{R4}] == 0)$, $\text{PC} \leftarrow (\text{PC} + 4) + \text{name}$; $-2^{15} \leq \text{name} < 2^{15}$

End of the lecture

Questions and exercises

- Suppose the following instructions are used in DLX architecture. State the meanings of the instructions, indicate their addressing modes, and articulate their encoding format.

LW R2, 100(R3)

SD 100(R3), F6

ADD R1, R2, R3

ADDI R1, R2, #10

BEQZ R5, 100

JR R2